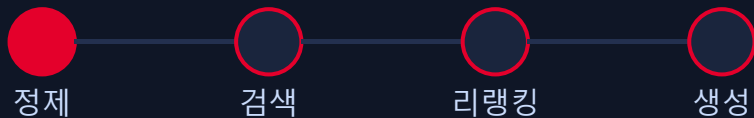


Copilot **Meta-Prompting**

Copilot Chat 프롬프트를 가로채 자동 정제하는 메타프롬프팅 시스템

@refine — 거친 한 줄 질문을, 검증된 고품질 프롬프트로



왜 메타프롬프팅인가

문제

- 1 LLM 답 품질은 프롬프트의 질에 좌우된다
좋은 답은 좋은 질문에서 나온다
- 2 하지만 사용자는 짧고 모호하게 묻는다
예) “정렬 코드 짜줘”
- 3 → 빗나간 답 → 다시 묻는 비효율
왕복이 늘수록 토큰·시간이 샌다

핵심 통찰

정답은 “더 비싼 모델”이 아니라 “더 좋은 프롬프트”다.

하위(소형) 모델도 잘 설계된 프롬프트를 받으면 고품질 결과를 낸다 → 토큰 비용 절감.

해법

사용자 입력과 LLM 사이에 ‘정제 계층’을 끼워 넣는다

단, 사용자 통제권을 지키려 보내기 전 미리보기·승인 게이트를 둔다.

“끼어들되 가로막지 않는다”

한 줄 정의와 핵심 가치

VS Code Copilot Chat에 **@refine**를 추가해, 명시 호출한 프롬프트만 메타프롬프팅으로 정제하고, 미리보기·승인 게이트를 거쳐 네이티브 Copilot에 전달하는 시스템.

품질

품질 향상

651개 검증 템플릿 기반 RAG로 프롬프트를 구조화·구체화해 재작성.

통제

사용자 통제

정제본을 눈으로 보고 5가지 선택지(전송·수정·재정제·원본·취소)로 승인.

안전

안전한 개입

fail-open — 서버 장애·예외에도 원본을 그대로 흘려보내 흐름을 막지 않음.

전체 구조 — FE 확장 ↔ BE 파이프라인



@refine 한 줄의 여정

- 1 명시 호출**
사용자가 @refine 헝가리의 수도가 어디야? 입력
- 2 정제 요청**
refiner.js → BE POST /refine {"prompt": ...}
- 3 미리보기 + 승인**
정제본 미리보기 + 가로 액션 바(아이콘 5개)
- 4 전송 위임**
승인 시 workbench.action.chat.open 으로 네이티브 Copilot 제출
- 5 기록 · 시각화**
allow 1건마다 JSONL append → 사이드바 카드 갱신(fs.watch)

승인 — 5가지 선택지

✓ 전송	정제본 자동 전송	기록
✎ 수정	입력창에 채워 편집	—
↶ 다시 정제	원본으로 재정제	—
→ 원본 전송	정제 없이 원본 전송	—
✕ 취소	@refine 원본 입력창 복원	—

정제는 어떻게 일어나는가 — 4단계



전체 파이프라인 = **POST /refine** · 4개 단계는 각각 단독 엔드포인트로도 호출·디버깅 가능

① VS Code Extension (FE) — swbc-prompt-refiner

@refine 참가자 + 5선택지 승인 게이트

정제 미리보기 → 전송·수정·다시정제·원본·취소 (codicon 액션 바)

fail-open + 명시 호출만

장애·예외에도 원본 전송 · @refine 현재 요청만 정제

정제 내역 로그 + 사이드바 시각화

allow 1건마다 JSONL append → fs.watch → 원본→정제본 카드

Chat Participant API · WebviewView · Node http/https · JSONL

요구사항

R-EX-01 ~ 11

0

런타임 의존성 (순수 JS)

25 / 25

단위 테스트 통과

v0.7.0

단일 .vsix (~23KB · 13파일)

② Endpoint — FastAPI 정제 API 서버

6개 엔드포인트

/health · /refine · /rewrite · /search · /rerank · /generate

/refine = 전체 파이프라인

나머지 4개는 각 단계 단독 실행 · 디버깅용 · /docs Swagger

웹 시각화 UI

파이프라인 단계 별 변화 + 벡터 DB 2D 지도(t-SNE·KMeans)

FastAPI · uvicorn · requests · **자족 모듈(retrieval.py + artifacts)**

요구사항

R-EP-01 ~ 02

6

REST 엔드포인트

.env

주소·모델만 주입 (하드코딩 0)

4단계

정제 → 검색 → 리랭킹 → 생성

③ Vector DB & Reranker — 검색·선별

Chroma 벡터 DB

production_task_prompt_templates 컬렉션 · 코사인 유사도 검색

템플릿 출처

GitHub 프롬프트 + selected_prompts.csv 정규화 적재

2단계 선별

임베딩 Top-20 → Reranker 의미 적합성 → Top-3

ChromaDB · Qwen3-Embedding-8B · Qwen3-Reranker-4B

요구사항

R-DB-01~02 · R-RK-01~03

651

고품질 프롬프트 템플릿

4096

임베딩 차원 (Qwen3-8B)

20 → 3

검색 → 리랭킹 선별

④ AI Infra — vLLM 모델 서빙

LLM (생성/정제)

cyankiwi/Qwen3.6-35B-A3B-AWQ-4bit · AWQ 4-bit MoE

Embedding / Reranker

Qwen3-Embedding-8B · Qwen3-Reranker-4B

배포 유연성

로컬 vLLM / RunPod 모두 지원 · .env로 주소·모델만 교체

vLLM 0.19.1 · OpenAI 호환 엔드포인트 · GPU별 서빙

요구사항

R-IN-01~02 · R-RW · R-FC

3

서빙 모델 (LLM·임베딩·리랭커)

4-bit

AWQ 양자화 (메모리 절감)

vLLM

고속 추론 서빙

레이어별 기술 스택

FRONTEND

VS Code 확장

- VS Code Extension API (^1.90)
- Chat Participant API
- Webview View API + CSP/nonce
- Node 내장 http / https / fs
- JSONL 로그 (append-only)
- @vscode/vsce (npm 패키징)
- 런타임 의존성 0 — 순수 JS

BACKEND

API · 파이프라인

- FastAPI 0.138 · uvicorn 0.49
- requests 2.34
- ChromaDB 1.5 (벡터 검색)
- OpenAI SDK 2.44 (호환 호출)
- scikit-learn (t-SNE · KMeans)
- 정적 웹 UI (HTML·JS·CSS)
- 버전 핀 고정 (.venv 기준)

AI / INFRA

모델 서빙

- vLLM 0.19.1
- Qwen3.6-35B-A3B (LLM, AWQ)
- Qwen3-Embedding-8B (4096d)
- Qwen3-Reranker-4B
- OpenAI 호환 /v1/* API
- AWQ 4-bit 양자화
- 로컬 GPU / RunPod 배포

흔들리지 않는 설계 불변식



fail-open

핸들러·정제·로깅 전부 try/catch. 어떤 에러에도 원본을 흘려보내 Copilot 흐름을 막지 않음.



승인 게이트

정제본은 사용자가 눈으로 보고 고른 뒤에만 전송 — 자동 치환 강요 없음.



JSONL 스키마 동결

사이드바가 읽는 데이터 계약. {ts·channel·model·before·after} 고정.



명시 호출만

@refine로 부른 현재 요청만 정제. 그 외 트래픽은 일절 건드리지 않음(무수정).



의존성 0 (FE)

외부 패키지 미사용 → 공급망 위험 없음 · 수십 KB .vsix · 단순한 리뷰.



사용자 데이터 보존

정제 관측 로그 observe*.log는 삭제·덮어쓰지 않음.

요구사항 구현 현황

모듈	요구사항	상태	근거
VS Code Extension	R-EX-01~11 (11)	구현	25 테스트 통과 (자동 10 · 사람검증 R-EX-08)
Endpoint (API)	R-EP-01~02 (2)	구현	FastAPI /refine 외 5개 엔드포인트
AI Infra (vLLM)	R-IN-01~02 (2)	구현	LLM·임베딩·리랭커 3모델 서빙
프롬프트 재작성	R-RW-01~05 (5)	구현	pipeline.rewrite (메타프롬프트 주입)
Vector DB	R-DB-01~02 (2)	구현	Chroma 651 템플릿 · Top-20 검색
Reranker	R-RK-01~03 (3)	구현	Qwen3-Reranker → Top-3 선별
최종 프롬프트 생성	R-FC-01~05 (5)	구현	pipeline.generate (최적 템플릿 합성)

합계 7개 모듈 · 30개 요구사항 전부 구현 (자동 검증 불가한 시각 비는 사람·시연 검증으로 명시)

테스트 & 신뢰성 — 25 / 25 통과

25

/ 25 통과

fail 0 · skipped 0
duration ≈ 185 ms



Node 내장 러너

node:test + node:assert로 실행 · 추가 의존성 0. JUnit XML 리포트로 요구사항 ↔ 테스트 ↔ 결과를 매칭.



커버리지

5개 액션 · 로그 append/read · fail-open(500 · 연결실패 · text-500 · 예외)까지 결정론적으로 검증.



결정론적 목(mock)

vscode-stub로 VS Code API 후킹 + 로컬 http 서버로 서버 응답 시뮬레이션 — 네트워크 비의존.



[R-EX-NN] 네이밍

모든 테스트 이름에 요구사항 ID 부여. R-EX-08(액티비티 바)은 시각 UI라 사람/시연 검증으로 구분.

DevOps & 실행 가능성



라이브러리 목록

package.json · requirements.txt (모듈별)



빌드 · 설치 문서화

RUN.md · build.sh (.vsix 패키징)



환경변수 문서화

.env.example (REFINE_API_URL · LLM/EMBED/RERANK)



빌드 성공

단일 .vsix 생성 (~23KB · 13파일)



버전 고정

백엔드·AI·확장 모두 핀 고정



실행 · 기동 문서화

F5 / uvicorn / serve_*.sh



외부 자원 안내

vLLM·RunPod 주소 · Chroma artifacts



애플리케이션 기동

F5 Extension Host / uvicorn 서버

결과파일은 program_extension · program_backend · program_AI로 분리 — src / tests / test-results 정합 구성

Git 협업 — 모듈별 브랜치 + PR

51

커밋

8

브랜치(모듈별)

10+

PR 머지

6

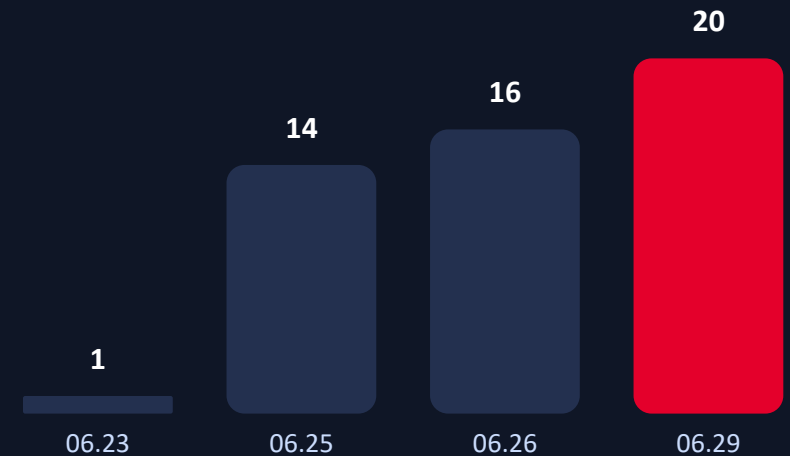
기여자

브랜치 = 모듈 · PR 기반 통합

extension · **endpoint** · **rag** · **LLM** · **infra** · **web** · **main**

- 모듈별 feature 브랜치에서 작업 후 Pull Request로 main에 통합 (#5 ~ #14).
- 마감 직전 물림 없이 2026-06-23 ~ 06-29 분산 작업.
- 결과파일에 Git 히스토리 포함 — 팀 단위 협업 지표 추적 가능.

일자별 커밋



결과 요약

한 주 만에 만든, 동작하는 메타프롬프팅 파이프라인

7 / 30

모듈 / 요구사항 구현

25 / 25

단위 테스트 통과

651 · 4096

템플릿 · 임베딩 차원

0

FE 런타임 의존성

요구사항 분석 → 설계 → 구현 → 테스트 → 실행까지 전 사이클 완결

FE 확장(가로채기·승인·시각화)과 BE 파이프라인(정제·검색·리랭킹·생성)을 모듈별 브랜치·PR로 통합했고, 검증 가능한 증거(테스트 로그 ·.vsix·git 히스토리)와 함께 7개 모듈 30개 요구사항을 구현했다.

한계 · 향후

남은 과제와 다음 걸음

한계

- 채팅 표면 스타일 제약 — 채팅 버튼은 커스텀 색.css 불가(테마색 고정). 브랜드 색은 사이드바 webview에서만.
- 정제 품질은 BE 의존 — 알고리즘·모델은 정제 서버 책임. 확장은 호출·게이트·기록만 담당.
- 외부 GPU 서빙 전제 — vLLM 모델은 로컬 GPU 또는 RunPod 엔드포인트 가동이 필요.

향후

- 정제 서버 주소를 VS Code 설정으로 노출 — 사용자가 직접 엔드포인트 전환.
- 모델 메타데이터 로깅 — 어떤 모델·템플릿으로 정제됐는지 기록·분석.
- 정제 프리셋 — 도메인별(코드·문서·번역) 정제 스타일 선택.
- 응답 캐싱 — 동일 입력 재사용으로 지연·토큰 절감.